

Project ATLAS — Day 2 Session Log: Detection Engineering

Step-by-step record of work performed, with rationale, for SC-200 study and SOC Analyst portfolio purposes

Ivan

June 27, 2026

Overview

Today's session closed out **Day 2 (Detection Engineering)** of Project ATLAS: a self-built SOC Tier 1 simulation using Microsoft Sentinel, Azure Monitor Agent (AMA), and a Sysmon-instrumented, Azure Arc-onboarded Windows 11 VM (DESKTOP-TRF9U79) as the simulated victim endpoint.

The session covered five things, in order:

1. Fixing a missing telemetry table (`SecurityEvent`) that was blocking detection content from working at all.
2. Enabling a Microsoft-built ("out-of-box") analytics rule.
3. Building a custom detection rule for brute-force logon activity.
4. Building a custom detection rule for encoded PowerShell execution.
5. A short, unplanned but valuable detour: auditing Microsoft Defender for Cloud's billing plans to make sure nothing was going to rack up cost overnight.

Each section below explains **what was done**, **how it was done**, and — most importantly for interview purposes — **why it was done**, tied back to real SOC Analyst responsibilities.

Part A — Restoring `SecurityEvent` Telemetry

The problem

Sentinel's built-in detection content for Windows authentication events (logon failures, account lockouts, privilege use, etc.) is written against a table called `SecurityEvent`. Querying it returned "table not found" — meaning no Windows Security log data was reaching Sentinel at all, even though Sysmon telemetry was already flowing into a different table (`Event`).

Why this happened

There are **two separate ingestion paths** for Windows Security event data into Azure Monitor Agent, and they are easy to confuse:

- A generic **Windows Event Logs (XPath)** data source, which routes everything into the generic `Event` table only.
- A dedicated **“Windows Security Events via AMA”** connector, which uses a special Microsoft-managed data stream (`Microsoft-SecurityEvent`) and is the *only* path that populates the legacy-schema `SecurityEvent` table that Sentinel’s prebuilt security-event detections expect.

The original Day 1 setup only configured the generic path, so `Event` had data but `SecurityEvent` did not exist.

What was done

1. Opened **Microsoft Sentinel → Content hub**, searched for and installed the **“Windows Security Events”** solution (a Microsoft-published content pack that ships the dedicated connector plus matching analytics rule templates).
2. Opened the newly installed **“Windows Security Events via AMA”** connector and used its **“+ Create data collection rule”** shortcut.
3. Built a new Data Collection Rule, `dcr-soc-atlas-security-events`, scoped to resource group `rg-soc-atlas`, with `DESKTOP-TRF9U79` as the only target resource.
4. On the “Collect and deliver” step, selected **AllEvents** (broader than the guide’s default “Common” event set) so that brute-force-relevant Event ID 4625 (failed logon) would definitely be captured.
5. Confirmed deployment validation passed and the DCR deployed successfully.

Why this matters

This is a realistic SIEM data-pipeline problem, not a lab artifact: telemetry “looking like it’s flowing” (data is arriving somewhere) while the *specific table a detection rule depends on* is empty is one of the most common root causes of “silent” detection gaps in production SOC’s. Diagnosing which connector feeds which schema, rather than assuming all log sources are interchangeable, is exactly the kind of telemetry/connector troubleshooting called out in the job description’s “analyze logs, alerts, and telemetry from security tools” and “improve detection, response, and alerting capabilities” responsibilities.

Status note

The table schema now exists and the DCR is deployed correctly. As of the end of this session, a `SecurityEvent | take 10` query had not yet returned rows – consistent with AMA’s first-ingestion lag (this environment has shown ingestion delays of up to ~2 hours before). **Action for next session:** re-run `SecurityEvent | take 10` to confirm rows are landing before relying on it for Day 3 attack-simulation validation.

Part B — Enabling a Built-In Analytics Rule

What was done

In the Defender portal's Sentinel **Analytics** → **Rule templates** tab, browsed the full template list (installed automatically by the Content Hub solution from Part A) and enabled **"SecurityEvent – Multiple authentication failures followed by a success"** — a Microsoft-authored detection for a classic brute-force-then-success pattern. Confirmed it now appears under **Active rules** with Status: Enabled.

A note on how the rule-template search actually works

Searching the template list for concept terms like "failed sign," "brute force," and "PowerShell" returned zero results. This isn't a bug — the search box does a literal substring match against rule *names* only, not a concept/semantic search, despite Microsoft's own guidance suggesting "search by concept." The fix was simply clearing the search box and scanning the full unfiltered list (23 templates) by eye.

Why this matters

Rule templates are inert blueprints — they only populate into a tenant after the relevant Content Hub solution is installed, and only become live, scheduled detections after you explicitly convert one ("Create rule"). Knowing this distinction (and knowing how to actually find relevant content in a sea of vendor-shipped templates) reflects the "govern and enhance core security services, including SIEM" and "lead or support the deployment, integration, and configuration of new security technologies" parts of the job description — recognizing and properly activating existing detection content is a real, recurring SOC engineering task, not just writing custom rules from scratch.

Part C — Custom Rule 4.1: Brute-Force Failed Logon Spike

The detection logic

```
SecurityEvent
| where TimeGenerated > ago(1d)
| where EventID == 4625
| summarize FailedAttempts = count(), Accounts = make_set(Account) by Computer, IpAddress
| where FailedAttempts >= 5
| order by FailedAttempts desc
```

This buckets failed logons (Event ID 4625) into 5-minute windows per host/source-IP pair and flags any window with 5 or more failures — a simple but realistic threshold-based brute-force detection.

MITRE ATT&CK mapping

Credential Access (T1110) → Brute Force, sub-technique T1110.001 (Password Guessing). Severity: Medium.

What was done

1. Created a new **Scheduled query rule** in Analytics, named ATLAS – Brute force failed logon spike.
2. Set the MITRE mapping above using the rule wizard's tactic/technique selector (a two-level UI: checking the parent tactic alone is not enough – the specific technique has to be expanded and checked separately via the > arrow next to the tactic).
3. Pasted and test-ran the query above in the wizard's "Advanced hunting" test panel (0 results, expected – no real attack traffic exists yet; that comes in Day 3).
4. Set the rule to run every 5 minutes, with no automated response action (intentionally skipped – automation comes later, once detections are proven).
5. Created the rule, then discovered the **Entity mapping** tab had been skipped.

The entity-mapping issue (and why it matters)

Sentinel's entity mapping requires *two* things per entity, not one: an abstract **Identifier** (e.g., for a Host: `HostName`, `FullName`, `NetBiosName`, etc.) and a **column mapping** pointing to the actual query field (e.g., `Computer`). Filling in only the column mapping – which looks complete – still throws a validation error, because Sentinel can't resolve what *kind* of identifier that column represents.

This was fixed after the fact via **Active rules → select rule → Edit → Entity mapping tab**, setting Host's Identifier to `HostName` (mapped to `Computer`) and IP's Identifier to `Address` (mapped to `IpAddress`), then saving – which updates the existing rule in place rather than duplicating it.

Entity mapping isn't cosmetic: it's what lets Sentinel auto-populate Host/IP/Account "entity" badges on the resulting incident, which is what powers incident correlation, automated playbooks, and analyst triage speed in a real SOC. A rule that fires but maps no entities still creates an incident – it's just a much weaker one to investigate. This directly reflects "improve detection, response, and alerting capabilities" and the investigation/triage side of the job description.

Part D – Custom Rule 4.2: Encoded PowerShell Execution

The detection logic

Event

```
| where TimeGenerated > ago(1d)
| where Source == "Microsoft-Windows-Sysmon" and EventID == 1
| extend Image = extract(@"Image:\s*(.*?)\r?\n", 1, RenderedDescription)
| extend CommandLine = extract(@"CommandLine:\s*(.*?)\r?\n", 1, RenderedDescription)
| extend User = extract(@"User:\s*(.*?)\r?\n", 1, RenderedDescription)
| where Image has_any ("powershell.exe", "powershell_ise.exe")
| where CommandLine has_any ("-enc", "-EncodedCommand", "-e ")
| project TimeGenerated, Computer, User, Image, CommandLine
```

This targets Sysmon Event ID 1 (process creation) where the process image is PowerShell and the command line contains an encoded-command flag – a textbook defense-evasion indicator, since

attackers base64-encode PowerShell payloads specifically to dodge plaintext keyword detection and casual log review.

Note the `extract()` calls: unlike Microsoft Defender for Endpoint's pre-parsed Advanced Hunting schema, Sysmon events land in the generic `Event` table as one long unstructured `RenderedDescription` blob, so the relevant fields (`Image`, `CommandLine`, `User`) have to be pulled out with regex first. This is a direct consequence of the Day 1 architecture decision to use Sysmon + AMA instead of Defender for Endpoint.

MITRE ATT&CK mapping

Execution (T1059.001 - PowerShell) + Defense Evasion (T1027 - Obfuscated Files or Information). Severity: High.

What was done

1. Created a new Scheduled query rule, `ATLAS - Encoded PowerShell execution`, with the MITRE mapping above.
2. Pasted and test-ran the query (0 results, expected for the same reason as 4.1 — no live attack yet).
3. Skipped automated response, same reasoning as 4.1.
4. Mapped entities (`Host` → `Computer`, `Account` → `User`) directly during creation this time — applying the lesson learned from the 4.1 entity-mapping miss.
5. Created the rule and confirmed it now appears in Active rules alongside 4.1, both showing Status: Enabled.

Why this matters

Dual-tactic mapping (Execution + Defense Evasion on one rule) demonstrates the kind of nuanced ATT&CK tagging real detection engineers do — most genuinely malicious behaviors map to more than one technique, and a SOC's detection coverage reporting (e.g., an ATT&CK Navigator heatmap) is only as accurate as how carefully each rule is tagged.

Part E — Verifying Both Rules Are Live

Confirmed both `ATLAS - Brute force failed logon spike` and `ATLAS - Encoded PowerShell execution` show **Status: Enabled** under Active rules, alongside the Part B out-of-box rule — three live detections total, all scoped to `law-soc-atlas`. Day 2's core detection-engineering objective is complete: detections exist and are scheduled before Day 3 generates any real attack telemetry to test them against.

Bonus: Cloud Cost Governance – Auditing Defender for Cloud Plans

What prompted this

Before stopping for the day, the natural question was: *is anything still going to bill while I'm away?* This isn't a throwaway question — knowing exactly what's running and what it costs is itself a real operational-security skill, separate from detection engineering.

What was checked, and where

Local VirtualBox VMs (Kali, Windows) only consume the Mac's own CPU/RAM — they never touch Azure billing regardless of power state. Log Analytics ingestion and Sentinel are covered under the active Azure free trial. The one real unknown was **Microsoft Defender for Cloud's paid plans**, which bill per-resource-hour continuously once turned on — independent of whether the resource is actively being used.

Checked via: **Azure Portal → Microsoft Defender for Cloud → Environment settings → (drilled into) Tenant Root Group → subscription → Defender plans.**

(Note: the unified Defender portal's left-nav "Cloud security" item looked like a shortcut to the same thing, but is actually a different, newer CNAPP onboarding flow that requires a separate multi-hour tenant-preparation step — not what was needed here. The classic Azure Portal path above is still the correct one for plan-level billing toggles.)

What was found

Every plan except one showed **0 resources** in scope — meaning regardless of their On/Off toggle state, they were costing nothing, since Defender for Cloud bills per actual resource, not per toggle. The exception: **Servers → Plan 2 (\$15/server/month) → 1 server (DESKTOP-TRF9U79) → toggled On.** This was the one real, continuously-billing item.

This had not been deliberately enabled — the most likely explanation is that either Foundational CSPM auto-enables on first visit to Defender for Cloud, or (more likely, given the timing) the Azure Arc "Add a single server" onboarding wizard from Day 1 Part F silently enabled enhanced monitoring as part of connecting the machine. (Azure Activity Log, filtered for the "Update security pricing" operation, would confirm the exact timestamp/trigger if this needs to be nailed down later.)

Action taken

Toggled the **Servers** plan to **Off** and saved, eliminating the one real billing risk before logging off for the day. All other plans were left as-is since they carry zero cost at 0 resources.

Why this matters for the role

This whole detour maps directly onto a specific line in the target job description: **"Balance security best practices with operational and business requirements."** Knowing which security tooling silently costs money, auditing it proactively, and making a deliberate on/off decision rather than just leaving everything maximally enabled is a real, recurring responsibility for anyone operating cloud security tooling at scale — not a detection-engineering skill, but a cost-governance one, and a good interview anecdote in its own right ("I noticed a paid plan had been silently enabled during

onboarding and audited the subscription to confirm it was the only billing exposure before disabling it”).

Status Snapshot

Item	Status
SecurityEvent table schema + DCR	Created; data ingestion pending re-check next session
Part B – built-in rule enabled	Complete
Part C – Rule 4.1 (brute force)	Complete, entity-mapped
Part D – Rule 4.2 (encoded PowerShell)	Complete, entity-mapped
Part E – both rules confirmed live	Complete
Defender for Cloud cost audit	Complete – Servers plan disabled, zero billing risk overnight

Next session

1. Re-run `SecurityEvent | take 10` to confirm Windows Security Event data is now flowing.
2. Move into **Day 3 – Attack Simulation**: Nmap recon, Hydra brute force, base64-encoded PowerShell reverse shell, phishing via Attack Simulator, and a Tor sign-in to trigger Entra ID Protection – the activity these three Day 2 detections are designed to catch.